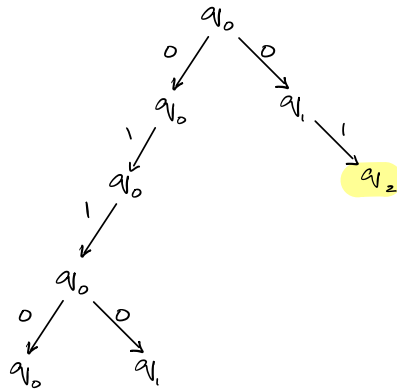




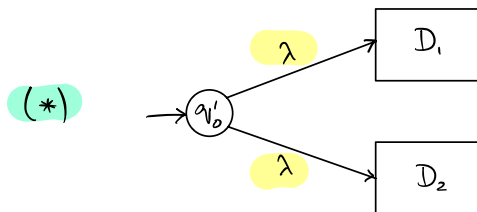
$w = 0110$ is not accepted.

Note path just dies off after only processing 01, since no transition on 1 from q_2 .

We have argued in class regular languages are closed under complement operation. What about union?

i.e., given two DFAs D_1, D_2 can we construct a DFA that accepts $L(D_1) \cup L(D_2)$?

Here is an NFA construction:



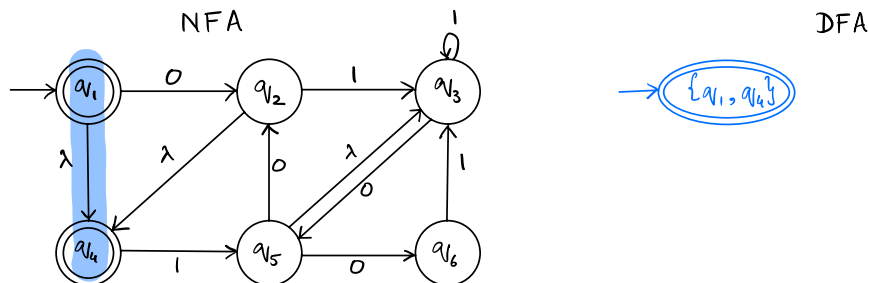
Add a new state q'_0 with λ -transitions to starting states of D_1, D_2 .

But this is not a DFA!
What about intersection?

Def An NFA N is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$

$\delta: Q \times (\Sigma \cup \{\lambda\}) \longrightarrow \mathcal{P}(Q)$
Powerset of Q .

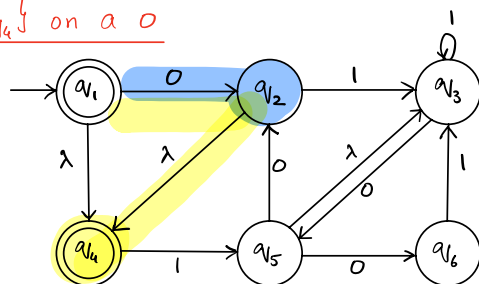
To complete our argument that class of languages accepted by DFAs are closed under union operation, we need to be able to convert the above construction (*) into a DFA. We illustrate this by running through the NFA to DFA conversion algorithm.



q_1, q_4 are reachable on λ . This determines the DFA starting state.
It is also an accepting state since both q_1, q_4 are accepting states.

Next we need to determine the DFA state $\{q_1, q_4\}$ transitions on 0 & 1.

$\{q_1, q_4\}$ on a 0



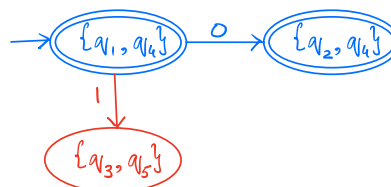
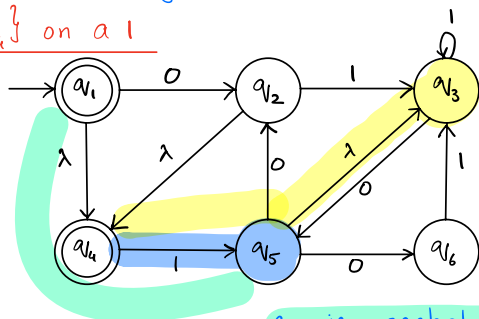
	0	1
$\{q_1, q_4\}$		



	0	1
$\{q_1, q_4\}$	$\{q_2, q_4\}$	

The highlighted transitions identify the two states $q_2 \neq q_4$ reachable from q_1 on a 0. There is no state reached from q_4 on a 0. So, $\{q_1, q_4\}$ transitions to $\{q_2, q_4\}$ on a 0. It is an accepting state since q_4 is an accepting state in the NFA.

$\{q_1, q_4\}$ on a 1



	0	1
$\{q_1, q_4\}$	$\{q_2, q_4\}$	$\{q_3, q_5\}$

q_3 is reachable from q_1 on a 1.

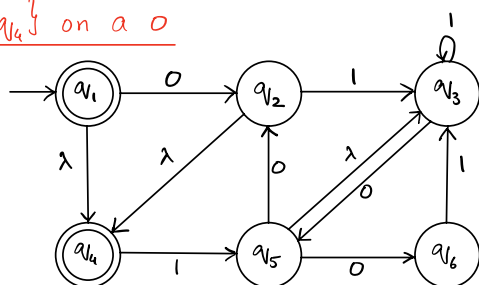
q_5 is reachable from q_4 on a 1.

q_3 is reachable from q_4 on a 1.

So, $\{q_1, q_4\}$ has a transition to $\{q_3, q_5\}$ on a 1. It is not an accepting state since both $q_3 \neq q_5$ are not accepting states.

Now, to complete the DFA we need to determine transitions of the new states $\{q_2, q_4\}$ & $\{q_3, q_5\}$.

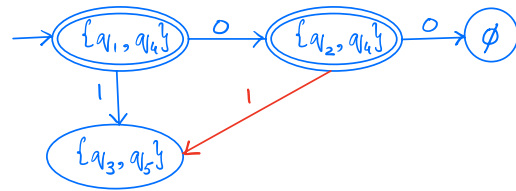
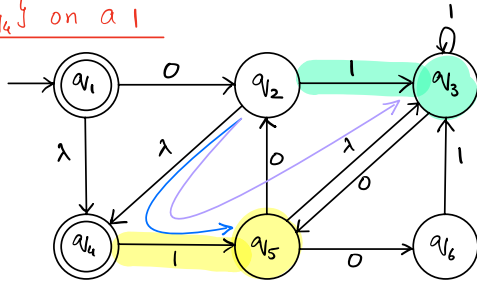
$\{q_2, q_4\}$ on a 0



	0	1
$\{q_1, q_4\}$	$\{q_2, q_4\}$	$\{q_3, q_5\}$
$\{q_2, q_4\}$	ϕ	
$\{q_3, q_5\}$		

There is **no** state reachable from either q_2 or q_4 on a 0. So, new DFA state is ϕ .

$\{a_2, a_4\}$ on a 1



	0	1
$\{a_1, a_4\}$	$\{a_2, a_4\}$	$\{a_3, a_5\}$
$\{a_2, a_4\}$	ϕ	$\{a_3, a_5\}$
$\{a_3, a_5\}$	ϕ	ϕ

On $\{a_2, a_4\}$
we reach
 $\{a_3, a_5\}$
on a 1.

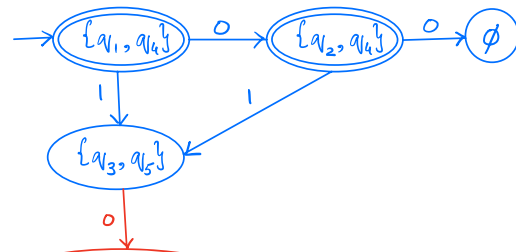
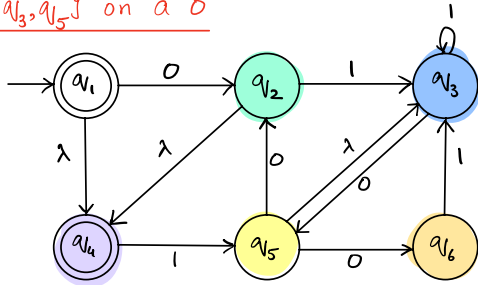
a_3 is reachable from a_2 on a 1.

a_5 is reachable from a_4 on a 1.

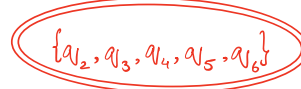
a_5 is reachable from a_2 on a 1.

a_3 is also reachable from a_2 on a 1 via $a_4 \rightarrow a_5$.

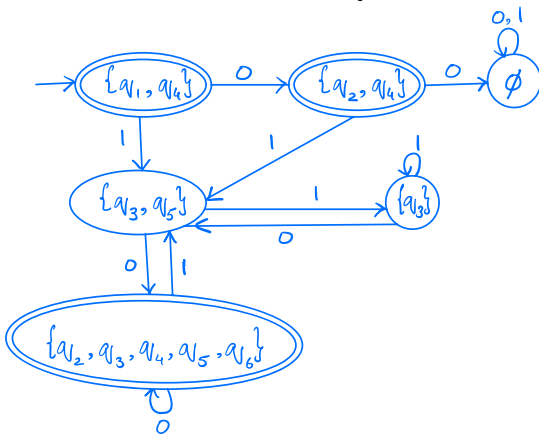
$\{a_3, a_5\}$ on a 0



$a_3 \rightarrow a_5$ on a 0.
 $a_3 \rightarrow a_5 \rightarrow a_3$ on a 0.
 $a_5 \rightarrow a_2$ on a 0.
 $a_5 \rightarrow a_6$ on a 0.
 $a_5 \rightarrow a_2 \rightarrow a_4$ on a 0.

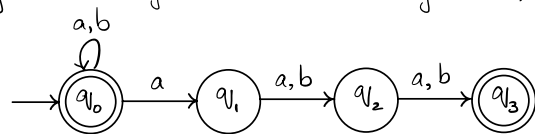


Similarly, continuing on, we conclude with following DFA.



	0	1
$\{a_1, a_4\}$	$\{a_2, a_4\}$	$\{a_3, a_5\}$
$\{a_2, a_4\}$	ϕ	$\{a_3, a_5\}$
$\{a_3, a_5\}$	$\{a_2, a_3, a_4, a_5, a_6\}$	$\{a_3\}$
ϕ	ϕ	ϕ
$\{a_2, a_3, a_4, a_5, a_6\}$	$\{a_2, a_3, a_4, a_5, a_6\}$	$\{a_3, a_5\}$
$\{a_3\}$	$\{a_3, a_5\}$	$\{a_3\}$

ex., Apply same algorithm to following NFA, $\Sigma = \{a, b\}$



Definition

Class of languages accepted by DFAs are called regular languages.

Since NFAs can be converted into equivalent DFAs, they also accept regular languages.

Therefore, construction (*) shows that regular languages are closed under union operation.

We have seen that the language

$\{w \in \Sigma^* \mid w \text{ has equal occurrences of } 01 \text{ and } 10\}$ is regular. However, it doesn't seem possible to design a DFA for the language

$\{w \in \Sigma^* \mid w \text{ has equal occurrences of } 0 \text{ and } 1\}$. Try it! Why?

We want to get a better understanding of what languages are not regular.

Note, all finite languages are regular. Why?

So, a non-regular language must be infinite. However, a regular language can also be infinite.

Key Observation

An automata for an infinite regular language must have a cycle in an accepting path.

$w = \underline{x}y\underline{z} \in L$



Pumping Lemma (for regular languages)

Let Σ be an alphabet & let $A \subseteq \Sigma^*$ be a regular language.

There exists a positive integer p , called the pumping length of A , with following property:

For every string $w \in A$, with $|w| \geq p$, there exists a decomposition of w into $w = xyz$, for $x, y, z \in \Sigma^*$, such that

- i) $y \neq \lambda$, $|y| \geq 1$,
- ii) $|xy| \leq p$, & iii) $w_i = xy^iz \in A$ for all $i \in \{0, 1, \dots\}$

Using the lemma to prove a language is not regular, e.g.,

$L = \{a^n b^n, n \geq 0\}$. Show L is not regular.

PROOF

Assume for sake of contradiction, L is regular. This implies L has a pumping length, let us say p .

We now need to choose a $w \in L$, with $|w| \geq p$. A reasonable choice is $a^p b^p$.

Let $w = a^p b^p \in L$, $|w| = 2p \geq p$.

We need to consider all decompositions of w into xyz , where $|xy| \leq p$ & $|y| \geq 1$. Note that xy can only be a string of a 's.

So, we have $x = a^l$, $l \geq 0$ & $y = a^k$, $1 \leq k \leq p$, $k+l \leq p$ (since $|xy| \leq p$)

This gives $w = xyz = \underbrace{a^l}_x \underbrace{a^k}_y \underbrace{a^{p-l-k} b^p}_z = a^{l+k+p-l-k} b^p = a^p b^p$

Consider $i=0 \Rightarrow w_0 = xy^0z = xz$ (since $y^0 = \lambda$) This is not $y = \lambda$
pumping down
 $= \underbrace{a^l}_x \underbrace{a^{p-l-k} b^p}_z$
 $= a^{p-k} b^p \notin L$ since $k \geq 1$.

This contradicts the pumping lemma. Hence our assumption is incorrect & L is not regular.

Similarly, we can also pump up, with $i=2$,

$$w_2 = xy^2z = xy yz = \underbrace{a^l}_x \underbrace{a^k}_y \underbrace{a^k}_y \underbrace{a^{p-k-l} b^p}_z = a^{p+k} b^p \notin L, \text{ since } k \geq 1.$$

How can you use above proof & closure properties to show

$\Sigma = \{a, b\}$, $L = \{w \in \Sigma^*, w \text{ has equal number of } a \text{ \& } b\}$ is not regular.

Try to show $L = \{a^m b^n, m > n\}$
 $L = \{w \in \Sigma^*, w = w^R\}$ are not regular using the pumping lemma.

PROOF OF PUMPING LEMMA

Let $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA that recognizes language A . Let $p = |Q|$ be the number of states of M .

Consider a string $w \in A$, with $|w| \geq p$. We have $w = w_1 w_2 \dots w_m$, $w_i \in \Sigma$
 $|w| = m \geq p$

Let $r_0, r_1, r_2, \dots, r_m$, $r_i \in Q$ be the sequence of states visited in the DFA M , when it processes string w . Note,

$r_0 = q_0$ (starting state) & $r_m \in F$ (accepting state)

$$r_{k+1} = \delta(r_k, w_{k+1}) \quad \begin{array}{l} \text{(sequence of states visited)} \\ k \in \{0, 1, \dots, m-1\} \end{array}$$

Since $m \geq p$, consider the sequence $r_0, r_1, \dots, r_p$
 $\underbrace{\hspace{10em}}_{p+1 \text{ states}}$

Recall the DFA M has only p states. (since $|Q| = p$) } Hence in the sequence we must revisit a state.

So, there must exist indices $s \neq t \in \{0, 1, \dots, p\}$, $s < t$, where $r_s = r_t$.

This allows us to fix

$$x = w_1 w_2 \dots w_s, \quad y = w_{s+1} \dots w_t, \quad z = w_{t+1} \dots w_p$$

string processed
in the cycle

The state that we revisit
in the cycle.

i) Since $t > s$, y cannot be λ .

ii) xy has length $t \leq t \leq p$ since $t \in \{0, 1, \dots, p\}$. So, $|xy| \leq p$

iii) Finally, $xy^i z \in A$ means M accepts $xy^i z$ for all $i \in \{0, 1, \dots\}$

Recall $\underbrace{r_0, r_1, \dots, r_s}_x, \underbrace{r_{s+1}, \dots, r_t}_y, \underbrace{r_{t+1}, \dots, r_m}_z$ is the accepting path.
 can repeat i times
 since $r_s = r_t$